

ANÁLISIS Y DISEÑO DE ALGORITMOS

Ramificación y poda

Práctica Final

Camino de coste mínimo
usando
Ramificación y poda

Nombre: Georg Usin
DNI/NIE: X8174555X



1. Estructura de datos	3
1.1 Nodo	3
1.2 Lista de nodos vivos	3
2. Mecanismos de poda	4
2.1 Poda de nodos no factibles	4
2.2 Poda de nodos no prometedores	4
3. Cotas pesimistas y optimistas	5
3.1 Cota pesimista inicial (inicialización)	5
3.2 Cota pesimista del resto de nodos	5
3.3 Cota optimista	5
4. Otros medios empleados para acelerar la búsqueda	6
4.1 Verificación de alcanzabilidad previa	6
4.2 Ordenación inteligente de direcciones	6
4.3 Memoización de estados	7
4.4 Representación linealizada del estado	7
5. Estudio comparativo de distintas estrategias de búsqueda	8
6. Tiempos de ejecución	11

1. Estructura de datos

1.1 Nodo

El nodo utilizado en la implementación contiene la siguiente información:

```
struct Nodo {
    int x, y;
    int longitud;
    shared_ptr<EstadoVisitado> visitado;
    shared_ptr<Nodo> padre; // Para reconstruir el camino

    // Para la cola de prioridad (menor longitud + heurística tiene prioridad)
    int cota_inferior() const {
        int dx = abs((n_filas-1) - x);
        int dy = abs((n_cols-1) - y);
        return longitud + max(dx, dy); //Distancia de Chebyshev
    }

    bool operator>(const Nodo& otro) const {
        return cota_inferior() > otro.cota_inferior();
    }

    // Constructor
    Nodo(int _x, int _y, int _longitud, shared_ptr<EstadoVisitado> _visitado, shared_ptr<Nodo> _padre = nullptr)
        : x(_x), y(_y), longitud(_longitud), visitado(_visitado), padre(_padre) {}
};
```

Componentes del nodo:

- **x, y**: Coordenadas actuales en la matriz del laberinto
- **longitud**: Longitud del camino desde el origen (número de celdas visitadas)
- **visitado**: Puntero compartido a la estructura que mantiene el estado de celdas visitadas
- **padre**: Puntero al nodo padre que permite reconstruir la solución óptima
- **cota_inferior()**: Función que calcula la cota optimista del nodo
- **operator>(const Nodo& otro)**: Función para comparar cotas inferiores entre nodos.
- **Nodo(int _x, int _y, int _longitud, shared_ptr<EstadoVisitado> _visitado, shared_ptr<Nodo> _padre = nullptr)**: constructor de clase.

1.2 Lista de nodos vivos

Se utiliza una **cola de prioridad** como estructura para mantener los nodos vivos:

```
// Cola de prioridad para ramificación y poda
auto comparador = [](const shared_ptr<Nodo>& a, const shared_ptr<Nodo>& b) {
    return *a > *b;
};
priority_queue<shared_ptr<Nodo>, vector<shared_ptr<Nodo>>, decltype(comparador)> cola_nodos(comparador);
```

Se extrae siempre el nodo con menor valor de cota inferior **longitud + max(dx, dy)**. Este criterio implementa una búsqueda **Least Cost** guiada por una heurística admisible.

2. Mecanismos de poda

2.1 Poda de nodos no factibles

Los nodos no factibles se descartan mediante las siguientes verificaciones:

```
// Verificar límites y factibilidad
if (ni < 0 || ni >= n_filas || nj < 0 || nj >= n_cols) {
    continue;
}

// Verificar que la celda sea transitable
if (matriz[ni][nj] == 0) {
    estadisticas.nodos_no_factibles++;
    continue;
}

// Verificar que no haya sido visitada previamente
if (actual->visitado->esta_visitado(ni, nj)) {
    continue;
}
```

Condiciones de no factibilidad:

1. Posición fuera de los límites del laberinto
2. Celda bloqueada (valor 0 en la matriz)
3. Celda ya visitada en el camino actual (evita ciclos)

2.2 Poda de nodos no prometedores

Un nodo se considera no prometedor en los siguientes casos:

```
// Poda: si ya tenemos una solución mejor
if (actual->longitud >= mejor_longitud) {
    estadisticas.nodos_no_prometedores++;
    continue;
}
```

```
// Poda por cota inferior
if (nuevo_nodo->cota_inferior() >= mejor_longitud) {
    estadisticas.nodos_no_prometedores++;
    continue;
}

// Poda por mejor distancia conocida
int pos_nueva = ni * n_cols + nj;
if (mejor_distancia[pos_nueva] <= nuevo_nodo->longitud) {
    estadisticas.nodos_prometedores_descartados++;
    continue;
}
```

Criterios de poda:

1. **Poda por cota superior:** Si la longitud actual $\geq$ mejor solución conocida
2. **Poda por cota inferior:** Si la estimación optimista $\geq$ mejor solución conocida
3. **Poda por memoización:** Si ya se visitó el estado con menor costo

3. Cotas pesimistas y optimistas

3.1 Cota pesimista inicial (inicialización)

La cota pesimista inicial se establece como el número de todas las casillas + 1

```
// Cota pesimista inicial
int mejor_longitud = n_filas * n_cols + 1;
```

3.2 Cota pesimista del resto de nodos

La cota pesimista se actualiza cada vez que se encuentra una solución completa:

```
// Si llegamos al objetivo
if (actual->x == n_filas - 1 && actual->y == n_cols - 1) {
    estadisticas.nodos_hoja++;
    if (actual->longitud < mejor_longitud) {
        mejor_longitud = actual->longitud;
        mejor_nodo_solucion = actual;
        estadisticas.mejor_solucion_desde_hoja++;
    }
    continue;
}
```

Cada vez que se alcanza el objetivo, se compara la longitud del camino con la mejor solución conocida y se actualiza si es mejor.

3.3 Cota optimista

La cota optimista para cada nodo se calcula mediante la distancia de Chebyshev:

```
int cota_inferior() const {
    int dx = abs((n_filas-1) - x);
    int dy = abs((n_cols-1) - y);
    return longitud + max(dx, dy); //Distancia de Chebyshev
}
```

La distancia de Chebyshev $\max(|dx|, |dy|)$ es admisible porque representa el mínimo número de movimientos necesarios para alcanzar el objetivo considerando que se pueden hacer movimientos diagonales.

4. Otros medios empleados para acelerar la búsqueda

4.1 Verificación de alcanzabilidad previa

```
// Comprueba si el destino es alcanzable usando búsqueda en anchura
bool es_alcanzable() {
    if (matriz.empty() || matriz[0].empty() || matriz[0][0] == 0 || matriz[n_filas-1][n_cols-1] == 0)
        return false;

    vector<bool> visitado_bfs(n_filas * n_cols, false);
    queue<pair<int, int>> cola;

    cola.push({0, 0});
    visitado_bfs[0] = true;

    while (!cola.empty()) {
        auto [i, j] = cola.front();
        cola.pop();

        if (i == n_filas - 1 && j == n_cols - 1)
            return true;

        for (int dir = 0; dir < 8; ++dir) {
            int ni = i + dx[dir];
            int nj = j + dy[dir];

            if (ni >= 0 && ni < n_filas && nj >= 0 && nj < n_cols &&
                matriz[ni][nj] == 1 && !visitado_bfs[ni * n_cols + nj]) {
                visitado_bfs[ni * n_cols + nj] = true;
                cola.push({ni, nj});
            }
        }
    }
    return false;
}
```

Su propósito es evitar exploración innecesaria verificando primero si existe al menos un camino al objetivo.

4.2 Ordenación inteligente de direcciones

```
// Función optimizada para obtener orden de direcciones hacia el objetivo
void inicializar_orden_direcciones(vector<int>& orden_dirs) {
    int objetivo_x = n_filas - 1;
    int objetivo_y = n_cols - 1;

    // Inicializa el orden por defecto
    for (int i = 0; i < 8; ++i) {
        orden_dirs[i] = i;
    }

    // Ordena las direcciones según la proximidad al objetivo usando heurística simple
    sort(orden_dirs.begin(), orden_dirs.end(), [objetivo_x, objetivo_y](int a, int b) {
        // Calcula qué dirección está más alineada con la dirección al objetivo
        int a_alineacion = (objetivo_x > 0 ? dx[a] : -dx[a]) + (objetivo_y > 0 ? dy[a] : -dy[a]);
        int b_alineacion = (objetivo_x > 0 ? dx[b] : -dx[b]) + (objetivo_y > 0 ? dy[b] : -dy[b]);
        return a_alineacion > b_alineacion;
    });
}
```

Para priorizar las direcciones que se acercan más al objetivo, mejorando la probabilidad de encontrar soluciones rápidamente.

4.3 Memoización de estados

```
// Tabla para evitar estados repetidos (memoización)
vector<int> mejor_distancia(n_filas * n_cols, numeric_limits<int>::max());
mejor_distancia[0] = 1;

if (mejor_distancia[pos_nueva] <= nuevo_nodo->longitud) {
    estadísticas.nodos_prometedores_descartados++;
    continue;
}

mejor_distancia[pos_nueva] = nuevo_nodo->longitud;
```

Evita re-explorar posiciones que ya fueron alcanzadas con menor costo.

4.4 Representación linealizada del estado

```
// Estructura para representar el estado visitado de forma linealizada
struct EstadoVisitado {
    vector<bool> visitado;

    EstadoVisitado(int n, int m) : visitado(n * m, false) {}

    bool operator[](pair<int, int> pos) const {
        return visitado[pos.first * n_cols + pos.second];
    }

    void marcar(int x, int y) {
        visitado[x * n_cols + y] = true;
    }

    bool esta_visitado(int x, int y) const {
        return visitado[x * n_cols + y];
    }
};
```

Optimiza el acceso a memoria usando indexación lineal en lugar de matriz bidimensional.

5. Estudio comparativo de distintas estrategias de búsqueda

	FIFO		LIFO		LC	
PRUEBA	nodos	tiempo	nodos	tiempo	nodos	tiempo
02	34	0.05	24	0.06	25	0.05
05	28	0.04	28	0.04	28	0.04
10	99	0.14	136	0.16	97	0.12
100	5948	11.36	18205	23.36	2284	5.56

FIFO (First-In, First-Out)

```
// VERSION FIFO
queue<shared_ptr<Nodo>> cola_fifo;
```

```
cola_fifo.push(inicial);
```

```
while (!cola_fifo.empty()) {
```

```
    auto actual = cola_fifo.front(); cola_fifo.pop();
```

```
    int inicio = 0, fin = 8, incremento = 1;
```

```
    cola_fifo.push(nuevo_nodo);
```

La estrategia FIFO garantiza la solución óptima debido a su naturaleza de búsqueda en amplitud. Sin embargo, esta característica también implica que explora muchos nodos innecesarios, especialmente en laberintos grandes. Esto se refleja en la prueba 100, donde se exploran 5.948 nodos. A pesar de ello, su rendimiento es razonablemente bueno en laberintos sencillos como en las pruebas 02, 05 y 10, donde tanto el número de nodos explorados como el tiempo de ejecución se mantienen bajos. Es una estrategia fiable pero ineficiente en escenarios complejos.

LIFO (Last-In, First-Out)

```
//VERSION LIFO  
stack<shared_ptr<Nodo>> pila_lifo;
```

```
pila_lifo.push(inicial);
```

```
while (!pila_lifo.empty()) {
```

```
    auto actual = pila_lifo.top(); pila_lifo.pop();
```

```
    int inicio = 7, fin = -1, incremento = -1;
```

```
    pila_lifo.push(nuevo_nodo);
```

La estrategia LIFO o búsqueda en profundidad, no garantiza la solución óptima y puede recorrer caminos muy largos antes de encontrar una solución. Esto se puede observar en la prueba 100, donde se exploran 18.205 nodos, una cifra muy superior a las otras estrategias. Aunque puede ofrecer tiempos aceptables en laberintos simples, su comportamiento empeora drásticamente en escenarios complejos, siendo la menos eficiente de las tres tanto en nodos explorados como en tiempo de ejecución.

LC (Least Cost)

```
// VERSION LC  
auto comparador = [](const shared_ptr<Nodo>& a, const shared_ptr<Nodo>& b) {  
    return *a > *b;  
};  
priority_queue<shared_ptr<Nodo>, vector<shared_ptr<Nodo>>, decltype(comparador)> cola_prioridad(comparador);
```

```
cola_prioridad.push(inicial);
```

```
while (!cola_prioridad.empty()) {
```

```
    auto actual = cola_prioridad.top(); cola_prioridad.pop();
```

```
    int inicio = 0, fin = 8, incremento = 1;
```

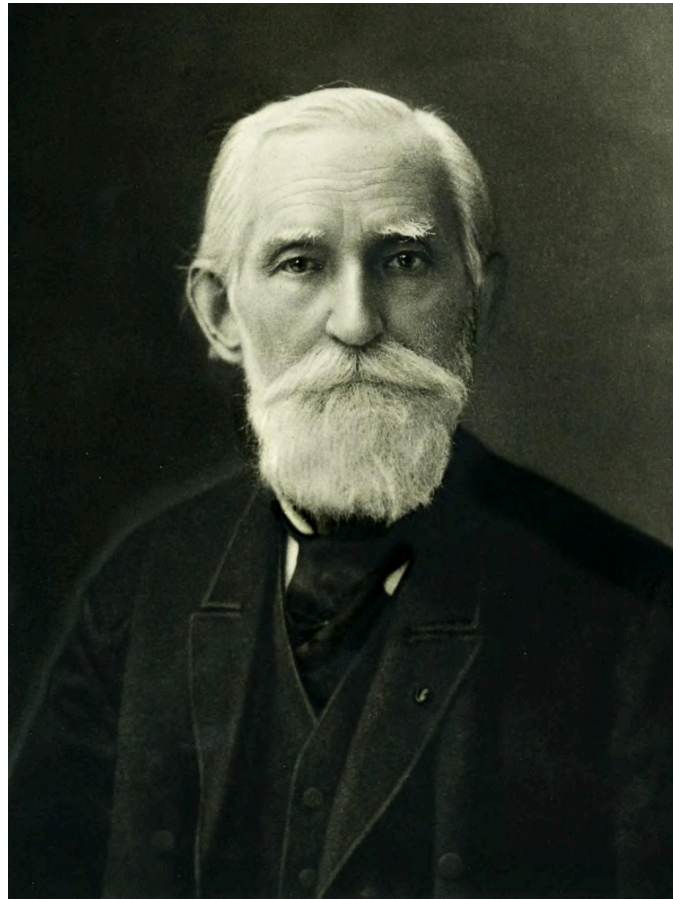
```
    cola_prioridad.push(nuevo_nodo);
```

La estrategia de búsqueda por costo mínimo (Least Cost) fue la más eficiente en términos de nodos explorados y tiempo total, especialmente en escenarios complejos. Por ejemplo, en la prueba 100 solo se exploraron 2.284 nodos, menos de la mitad de los que exploró FIFO y casi una octava parte de los de LIFO, con un tiempo de ejecución mucho menor. La principal desventaja de esta estrategia es que requiere una heurística para estimar el costo restante, lo que implica un coste computacional adicional. Sin embargo, esta inversión es muy rentable por la gran cantidad de los nodos explorados y el tiempo total.

Para la implementación de la estrategia Least Cost se eligió la **heurística de Chebyshev**, ya que se adapta mejor a la naturaleza del problema. En el laberinto considerado se permiten 8 movimientos (verticales, horizontales y diagonales), todos con el mismo coste. La distancia de Chebyshev considera el máximo entre la diferencia en filas y columnas, lo que permite estimar correctamente el número mínimo de pasos necesarios, incluso considerando movimientos diagonales.

También se probaron otras heurísticas como la distancia de Manhattan, la distancia de Octile y la distancia Euclidiana, pero debido a sus características que tan bien se adaptan a nuestro problema, la heurística de Chebyshev es la que mejores resultados dio.

Finalmente, la estrategia seleccionada para la resolución de la práctica ha sido **Least Cost** con la heurística de **Chebyshev**.



Pafnuty Chebyshev

6. Tiempos de ejecución

Fichero de test	Tiempo (ms)
100-bb.maze	5.56
200-bb.maze	24.89
300-bb.maze	177.14
500-bb.maze	385.02
700-bb.maze	2916.02
900-bb.maze	7087.90
k01-bb.maze	7265.24
k02-bb.maze	?
k03-bb.maze	?
k05-bb.maze	?
k10-bb.maze	?

7 de las 11 pruebas se solucionan correctamente en un tiempo inferior a los 15.000 milisegundos, las 4 pruebas restantes finalizan con un timeout.